



Dynamic RRT: Fast Feasible Path Planning in Randomly Distributed Obstacle Environments

Penglei Zhao¹ · Yinghui Chang² · Weikang Wu² · Hongyin Luo¹ · Zhixin Zhou¹ · Yanping Qiao¹ · Ying Li¹ · Chenhui Zhao¹ · Zenan Huang¹ · Bijing Liu¹ · Xiaojie Liu¹ · Shan He¹ · Donghui Guo¹

Received: 18 June 2022 / Accepted: 31 January 2023 / Published online: 24 March 2023
© The Author(s), under exclusive licence to Springer Nature B.V. 2023

Abstract

For path planning problems based on Rapidly exploring Random Trees (RRT), most new nodes merely explore the environment unless they are sampled directly from the subset that can optimize the path. This paper proposes the Dynamic RRT algorithm, which aims to plan a feasible path while balancing the convergence time and path length in an environment with randomly distributed obstacles. It estimates the length of a path from the start node to the goal node that is constrained to pass through an extended tree node, and this path length is heuristically taken as the major axis diameter of the informed subset. Then new node sampling is performed directly in this subset to optimize the estimated path. In addition, the idea of dynamic programming is employed to decompose the planning problem into subproblems by updating the node selected through Pareto dominance as the new start node to optimize the distance to the goal. Simulation results confirm the performance of the proposed algorithm in balancing the convergence time and path length and demonstrate that the convergence time is faster than that of RRT, while the path length is better than that of RRT*. Dynamic RRT also shows better performance than Lower Bound Tree-RRT(LBT-RRT), and Informed RRT* takes more time to compute a path of the same length.

Keywords Rapidly exploring Random Trees (RRT) · Dynamic RRT · Informed Subset · Dynamic Programming · Pareto Dominance

1 Introduction

In recent years, intelligent robots have proven to be efficacious for improving efficiency and automation in many tasks [1–3] and higher requirements are put forward [4–6]. Consequently, research on path planning algorithms for robot is extraordinarily vital [7, 8]. In particular, considering that the real-time speed and stability demands in these situations are usually high, the speed and stability of a planning algorithm are especially important [9].

Generally, path planning is a procedure in which a robot automatically finds a series of collision-free configurations between a given start node and a given goal node, mainly utilizing a grid-based or sampling-based method [10, 11]. Recently, methods based on reinforcement learning and

neural networks have also been investigated [12–14]. However, because of the exponential increase in the computational complexity of grid-based methods in high dimensions, methods based on reinforcement learning and neural networks converge increasingly slowly as the problem scale increases, making them inapplicable for high-dimensional and wide-ranging planning problems. In contrast, sampling-based methods can approach the target through fewer sample nodes and thus show prominent advantages in high dimensions. The most representative sampling-based algorithms for this purpose are Rapidly exploring Random Trees (RRT) and Probability Roadmaps (PRM) [15, 16]. PRM first constructs an undirected graph in the configuration space and then employs a path search algorithm such as A* to find a feasible path; this approach makes this algorithm unsuitable for dynamic scenarios, and it can easily fail under certain conditions [17, 18]. In contrast, RRT not only offers guaranteed effectiveness and robustness but is also easy to implement. Furthermore, it is easy to generalize to more complex situations. The aforementioned advantages make RRT popular in research on and applications of high-dimensional path planning [19].

✉ Donghui Guo
dhguo@xmu.edu.cn

¹ School of Electronic Science and Engineering, Xiamen University, Xiamen 361005, China

² Academy for Network & Communications of CETC, Shijiazhuang 050299, China

Given the superiority of RRT, it has attracted widespread attention since it was first proposed by S. M. Lavalle in 1998 [15]. However, since the new node generation process in RRT is entirely random, it can lead to problems such as narrow passages, poor reproducibility, slow convergence, and unsmooth paths. Accordingly, to overcome these deficiencies, various algorithm improvements have been introduced. Wei Wang et al. proposed an improved Bridge Test algorithm to identify narrow passages and then employed an online ϵ -greedy method to learn the extension of the tree nodes, which dramatically improved the performance of path planning in the presence of narrow passages [20]. To address the problem of slow convergence, the RRT-bias and extended RRT algorithms have been proposed, in which a new node steers either toward previously stored waypoints or directly toward the goal with a certain probability [21–23]. To mitigate the problem that the generated path tends to be unsmooth, in [24], Changbae Moon et al. used both a state tree and a workspace tree; the workspace tree was utilized to generate an initial path, which was then smoothed based on the state tree.

Since RRT has been proven to be nonoptimal, Karaman et al. proposed the RRT* algorithm to ensure that the generated path is asymptotically optimal [25]. On the one hand, as an optimized variant of RRT, RRT* imposes constraints on rewiring and reselecting the parent node when extending new nodes, thereby guaranteeing that each node is locally optimal. On the other hand, these modifications diminish the convergence speed compared with basic RRT. In past decades, to improve the performance of the RRT* algorithm and make it adapt to real-time planning, a series of improvements have been proposed. Article [26] proposed a new method that incorporates a dual-tree structure into RRT*, divides the whole process into path generation and path optimization, and improves the performance compared with the previous method. Gammell et al. proposed Informed RRT*, in which the sampling of new nodes is directly limited to the informed subset to optimize the path length. This approach shortens the convergence time while guaranteeing probabilistic completeness and asymptotic optimality, as in RRT*. However, this method degenerates to RRT* in the absence of prior knowledge [27].

From the above analysis, it can be concluded that current RRT methods are still unable to strike a proper balance between the convergence time and path length. Although Oren Salzman et al. proposed the Lower Bound Tree-RRT (LBT-RRT) algorithm, which can plan a higher-quality path than RRT within a shorter convergence time than RRT* [28]. Its performance is still insufficient. To overcome this problem, this paper proposes the Dynamic RRT algorithm, which achieves a significantly improved convergence time while guaranteeing the length of the generated path under the condition of a random obstacle distribution. The main contribution of this paper is described as follows:

- 1) First, this paper adopts the informed subset proposed in Informed RRT* [27]. However, instead of computing an initial path and then optimizing it as in Informed RRT*, the Dynamic RRT algorithm estimates the length of a path that is constrained to pass through an extended tree node. It then heuristically limits the size of the informed subset to a hyper-ellipsoid specified based on the estimated path length, from which new nodes can then be sampled to optimize the estimated path, thus improving the efficiency of new node generation.
- 2) Second, this paper incorporates the idea of dynamic programming. The concept of Pareto dominance as described in [29] is employed to select a Pareto-optimal node as the new start node and thus decompose the problem into subproblems so as to make the distance to the goal shorter, thereby improving the convergence time of the algorithm. Meanwhile, the Pareto dominance principle also ensures that each subproblem is locally optimal.
- 3) In this way, the Dynamic RRT algorithm can achieve a balance between convergence time and path length and can also provide strong stability and robustness.

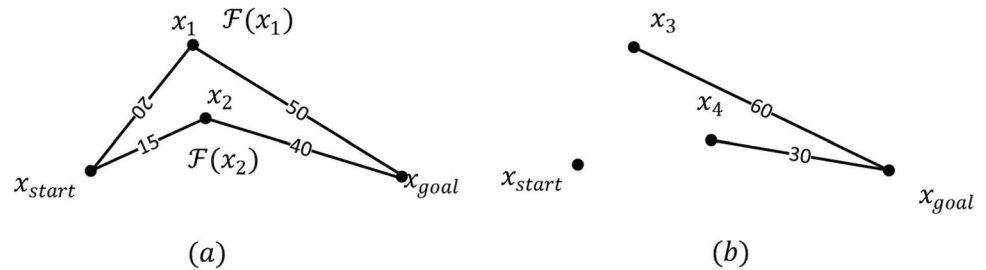
The remainder of this paper is structured as follows: Sect. 2 presents the definition of the path planning problem and introduces related terms. Then, Sect. 3 describes the improvements of the Dynamic RRT algorithm and gives its pseudocode. In Sect. 4, MATLAB simulations are reported to demonstrate the performance of Dynamic RRT in balancing the convergence time and path length. Finally, in Sect. 5, conclusions are given, and future research goals are specified.

2 Problem statement and notation

2.1 Problem statement

Generally, path planning problems include static and dynamic path planning. Static path planning can be defined as finding a collision-free path that connects a given start node and a given goal node in a known working environment. In contrast, dynamic path planning refers to generating an initial path and then adopting additional procedures and sensors to dynamically avoid newly detected obstacles [17, 29]. Notably, static planning is required in both cases; thus, static planning can be considered more crucial, although dynamic planning is more complex. Consequently, this paper mainly investigates static planning, and for convenience, unless otherwise specified, the path planning problem discussed below corresponds to static planning.

A workspace $X \in \mathbb{R}^m$ and a configuration space (C-space) $C \in \mathbb{R}^n$ are adopted to describe the planning problem, where m represents the dimensionality of the workspace, n represents

Fig. 1 Illustration of problem statement

the dimensionality of the configuration space, and X and C represent the Cartesian workspace and the configuration space, respectively. Equivalent to the workspace, the C-space also contains all relevant information of the working environment, including collision-free regions, obstacles, and boundaries. Therefore, for simplicity, the workspace is taken as an example below, but the presented discussion also applies to the C-space.

Let $x \in X$ be a state in the workspace; X_{free} is defined to represent the collision-free regions, X_{obs} to represent obstacles and boundaries, and $X_{total} = X_{free} \cup X_{obs}$ to represent the total space consisting of X_{free} and X_{obs} . In addition, $x_{start}, x_{goal} \in X_{free}$ are defined to represent the start and goal nodes, respectively, in the workspace. Let Σ be the set of all paths, and let Σ_{free} represent the set of all collision-free paths. Then the path planning problem can be described as

$$\delta(t) = \{ \delta(0) = x_{start}, \delta(1) = x_{goal}, \forall t \in [0, 1], \delta(t) \in X_{free} \} \quad (1)$$

where $\delta \subset \Sigma_{free}$ denotes the generated path and it is ensured that $\delta(t) \in X_{free}$ for any $t \in [0, 1]$, which means that find a feasible path, $\delta(t) : t \in [0, 1] \rightarrow X_{free}$ such that $\delta(0) = x_{start}$ and $\delta(1) = x_{goal}$, in a finite time; thus, it can also be called feasible path planning.

Notably, it has been proven that RRT satisfies probabilistic completeness and that RRT* satisfies asymptotic optimality on the basis of RRT [25]. These results mean that RRT converges quickly but cannot guarantee the path length, while the opposite is true of RRT*.

Definition 1. (Probabilistic completeness): For any path planning problem, if a feasible solution exists, as the number of iterations $i \in \mathbb{N}$ tend toward infinity, an algorithm can always find a solution such that.

$$P \left(\left\{ \begin{array}{c} \exists x_{start}, x_{goal} \in X_{free} \\ \cap \\ \lim_{i \rightarrow \infty} (V_i \cap x_{goal}) \neq \emptyset \end{array} \right\} \right) = 1 \quad (2)$$

where $\mathbb{N}$ represents the set of natural numbers and $V_i \subset X_{free}$ denotes the extended node tree.

Definition 2. (Asymptotic optimality): For any feasible path planning problem, the cost function, $\mathcal{C} : \Sigma_{free} \rightarrow \mathbb{R}_{\geq 0}$, of the path will always converge to a finite value $\mathcal{C}_{best} \in \mathbb{R}_{\geq 0}$ as the number of iterations $i \in \mathbb{N}$ tend toward infinity, such that.

$$P \left(\lim_{i \rightarrow \infty, \delta \in \Sigma_{free}} \mathcal{C}_i^\delta = \mathcal{C}_{best} \right) = 1 \quad (3)$$

where $\mathbb{R}_{\geq 0}$ represents the set of nonnegative real numbers.

Given that RRT is not guaranteed to generate a high-quality path as RRT* is, sampling directly from the informed subset is a better choice to improve the quality of the new nodes selected to extend the tree as well as the efficiency of their selection, as described in [27]. Thus, determining this subset is a necessary condition. As illustrated in Fig. 1, $\mathcal{F}(x)$ denotes the cost of a path from x_{start} to x_{goal} that passes through a specified node $x \in X_{free}$, which is equal to the sum of the numbers on the black lines. The numbers on the black lines represent the path lengths between two nodes, and $x_1, x_2 \in X_{free}$ are two candidate nodes, as shown in Fig. 1a. Accordingly, we will choose x_2 because $\mathcal{F}(x_2) = 55$ and $\mathcal{F}(x_1) = 70$, which means that $\mathcal{F}(x_2) < \mathcal{F}(x_1)$. However, since $\mathcal{F}(x)$ is generally unknown, an effective estimate $\mathcal{F}(x)$ is a necessary condition as well. Certainly, this will bring the convergence time of RRT closer to that of RRT*. Moreover, we can treat it as a dynamic programming problem and decompose it into subproblems to further improve the convergence time; for this purpose, efficiently choosing the intermediate node is also a necessary condition (Fig. 1b). $x_3, x_4 \in X_{free}$ denote the nodes in the node tree. The numbers on the black lines represent the Euclidean distance to x_{goal} . It can be seen that x_4 , with its value of 30, is a better choice because it effectively reduces the problem size.

2.2 Notation

In addition to the above definitions, the following terms are also adopted to introduce the algorithm and the improvements proposed in this paper.

CalCostHat: Given the start and end nodes, $x_{start}, x_{goal} \in X_{free}$, the extended node $x_{cur} \in X_{free}$, and the path cost $c_{cur} \in \mathbb{R}_{\geq 0}$ from x_{start} to x_{cur} , the function $\text{CalCostHat}(x_{start}, x_{goal}, x_{cur}, c_{cur})$

returns an estimate of the path length from x_{start} to x_{goal} , $c_{hat} \in \mathbb{R}_{\geq 0}$, to be applied as the major axis of the hyper-ellipsoid to restrict the informed subset.

SampleEllipsoid: Given the two focal points, $x_a, x_b \in X_{free}$, the total workspace X , and the major hyper-ellipsoid axis $c_{hat} \in \mathbb{R}_{\geq 0}$, the function $SampleEllipsoid(x_a, x_b, X, c_{hat})$ returns a randomly sampled node from the state space, $x_{rand} \in X_{free}$, through which the estimated cost from x_a to x_b is shortened compared with c_{hat} . Typically, the focal points are x_{start} and x_{goal} , and c_{hat} is calculated in advance via the $CalCostHat$ function.

Nearest: Given the node tree V_i extended by RRT and a random node x_{rand} , the function $Nearest(V_i, x_{rand})$ returns the nearest node to x_{rand} in V_i , $x_{nearest} \in X_{free}$.

Steer: Given two nodes x_{rand} and $x_{nearest}$, and the step size, $s \in \mathbb{R}_{\geq 0}$, the function $steer(x_{nearest}, x_{rand}, s)$ returns a new node $x_{new} \in X_{free}$ steering from $x_{nearest}$ to x_{rand} with step size s , only when the return value of the $CheckCollision$ function is false.

CheckCollision: Given two nodes $x_{nearest}$ and x_{new} , the function $CheckCollision(x_{nearest}, x_{new})$ returns a Boolean value representing whether the line from $x_{nearest}$ to x_{new} is feasible. A return value of true indicates a collision, meaning that x_{new} cannot be appended to V_i .

LineDistance: Given two arbitrary nodes, $x_1, x_2 \in X_{total}$, the function $LineDistance(x_1, x_2)$ returns the Euclidean distance from x_1 to x_2 .

Cost: Given the start node x_{start} and an arbitrary node $x_1 \in V_i$, the function $Cost(x_{start}, x_1)$ returns the path length from x_{start} to x_1 .

ChooseNodePareto: Given the goal node x_{goal} , the probability p , the estimated path length $\hat{\mathcal{F}}(x)$ and the node tree V_i , the function $ChooseNodePareto(V_i, \hat{\mathcal{F}}(x), x_{goal}, p)$ returns the next x_{start} candidate.

3 Proposed methods

This section describes the Dynamic RRT algorithm in detail. Section 3.1 introduces the estimation of the path length through extended nodes, which is used to heuristically limit the sampling range. In Sect. 3.2, the decomposition of the planning problem using Pareto dominance is presented. Finally, the pseudocode of the Dynamic RRT algorithm is provided, and a brief summary is given in Sect. 3.3.

3.1 Heuristic sampling range limits

Given a cost function $\mathcal{C}$, let $\mathcal{C}_{ab}$ denote the path length from a to b . Accordingly, the cost for a path from the x_{start} to x_{goal} constrained to pass through a specified node $x \in X_{free}$, $\mathcal{F}(x)$, can be defined as

$$\mathcal{F}(x) = \mathcal{C}_{sx} + \mathcal{C}_{xg} \quad (4)$$

where $\mathcal{C}_{sx}$ represents the path length from x_{start} to x and $\mathcal{C}_{xg}$ represents the path length from x to x_{goal} . Let $\mathcal{G}(x)$ and $\mathcal{H}(x)$

represent $\mathcal{C}_{sx}$ and $\mathcal{C}_{xg}$ respectively, such that Eq. 4 can be expressed as

$$\mathcal{F}(x) = \mathcal{G}(x) + \mathcal{H}(x) \quad (5)$$

For path planning problems, considering that the distribution of obstacles is generally unknown, virtual obstacles can be artificially added to the environment following a random distribution. Suppose that the obstacle distribution is a random variable $\Psi \in \mathbb{R}^m$ that obeys the distribution $\Psi \sim U(G)$, where $U()$ represents a random distribution; then, its probability density can be expressed as

$$f(\psi) = \frac{1}{S} \quad (6)$$

where G denotes the working environment, which is a bounded region in m -dimensional space; m denotes its dimensionality; and S denotes the volume of G .

When the node tree V is extended to node c , the path length from x_{start} to c , $\mathcal{G}(c)$, can be expressed as

$$\mathcal{G}(c) = \alpha \cdot \iint_{G_{sc}} f(\psi) d\psi \quad (7)$$

where $\psi \in \mathbb{R}^m$, α is a constant representing a scale factor, and G_{sc} denotes the bounded region containing δ_{sc} that represents the path from x_{start} to c . Considering that the obstacles are randomly distributed in region G , the selection of G_{sc} has no effect on the result. To facilitate the calculation, the minimum bounding rectangle (MBR) of δ_{sc} is selected. Thus, $\mathcal{G}(c)$ can be expressed as

$$\mathcal{G}(c) = \alpha \cdot \iint_{G_{sc}} f(\psi) d\psi = \alpha \cdot \frac{S_{sc}}{S} = \frac{\alpha}{S} \cdot \prod_{i=1}^m \lambda_i \quad (8)$$

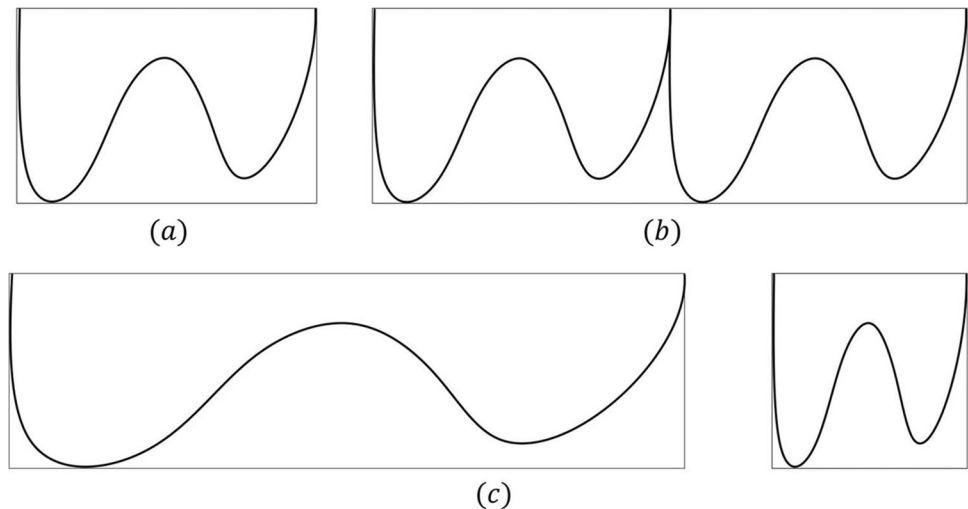
where S_{sc} denotes the volume of the m -dimensional hyperrectangle that contains δ_{sc} and λ_i denotes the length of its edges.

Similarly, the path cost from c to x_{goal} , $\mathcal{H}(c)$, can be expressed as follows

$$\mathcal{H}(c) = \beta \cdot \iint_{G_{cg}} f(\psi) d\psi \quad (9)$$

where β denotes a scale factor and G_{cg} denotes the bounded region containing δ_{cg} . However, since δ_{cg} does not exist, a precise admissible estimate of $\mathcal{H}(c)$, denoted by $\hat{\mathcal{H}}(c)$, must be obtained. Given that randomly distributed environment, the homotopic path of δ_{sc} , whose MBR is proportional in only one dimension, can be used as an acceptable estimate of δ_{cg} , as shown in Fig. 2. Then, $\hat{\delta}_{cg}$ can be estimated by repeating and scaling the path δ_{sc} in one dimension in accordance with the ratio of the Euclidean distance between them. Thus, $\hat{\mathcal{H}}(c)$ can be expressed as.

Fig. 2 Illustration of a path and its MBR: (a) presents a path δ_{sc} and its MBR; (b) presents the repeat of δ_{sc} and its MBR; (c) presents the scale of δ_{sc} and its MBR



$$\hat{\mathcal{H}}(c) = \frac{\beta}{S} \cdot \prod_{i=1}^m \mu_i \quad (10)$$

where μ_i denotes the length of each edge of the hyperrectangle containing δ_{sc} and $\forall i = 2, \dots, m, \mu_i = \lambda_i$. In this case, the scale factors α and β are equal to each other, such that

$$\hat{\mathcal{H}}(c) = \mathcal{G}(c) \cdot \prod_{i=1}^m \frac{\mu_i}{\lambda_i} = \mathcal{G}(c) \cdot \frac{\mu_1}{\lambda_1} \quad (11)$$

Accordingly, the cost of a path from x_{start} to x_{goal} that is constrained to pass through node c can be finally estimated as

$$\hat{\mathcal{F}}(c) = \mathcal{G}(c) \cdot \left(\frac{\mu_1}{\lambda_1} + 1 \right) \quad (12)$$

The pseudocode is shown in Algorithm 1. By convention, when c is equal to x_{start} , $\hat{\mathcal{F}}(c)$, is infinite.

Algorithm 1 Calculation of the estimated path length

Input: x_{start} : the start node of the planning problem;
 x_{goal} : the end node of the planning problem;
 x_c : the node selected as node c to estimate the path length;
 $\mathcal{C}_{sc}$: the path length from x_{start} to x_c ;

Output: $\hat{\mathcal{F}}(c)$: the estimated path length from x_{start} to x_{goal} passing through x_c ;

1. $\lambda_1 \leftarrow \text{LineDistance}(x_{start}, x_c)$
2. $\mu_1 \leftarrow \text{LineDistance}(x_{goal}, x_c)$
3. **if** $\lambda_1 \leq 0$
4. $\hat{\mathcal{F}}(c) \leftarrow \infty$
5. **else**
6. $\hat{\mathcal{F}}(c) \leftarrow \mathcal{C}_{sc} \cdot \left(\frac{\mu_1}{\lambda_1} + 1 \right)$
7. **end if**

For path planning problems, to ensure that the path cost to x_{goal} can be optimized to the greatest extent possible in each iteration, reference [27] notes that the probability of optimizing the path in any iteration by uniformly sampling from a larger subset becomes arbitrarily small and that sampling directly from the informed subset is a more effective method.

Definition 3. (informed subset): Given a path $\hat{\delta}_{sg}$ estimated as described above, the informed subset is the node set $\zeta(x) \supseteq \hat{\delta}_{sg}$ that can optimize the path to obtain a smaller cost, such that

$$\zeta(x) = \left\{ x \in X_{free} : \mathcal{G}(x) + \mathcal{H}(x) \leq \hat{\mathcal{F}}(c) \right\} \quad (13)$$

where x is the node to which the tree is to be extended. Considering that $\mathcal{G}(x)$ and $\mathcal{H}(x)$ are unknown, let $\hat{\mathcal{G}}(x)$ and $\hat{\mathcal{H}}(x)$ represent admissible heuristics for $\mathcal{G}(x)$ and $\mathcal{H}(x)$, respectively. As mentioned above, the Euclidean distance and the probability distribution of the obstacles can be employed to accurately estimate both of these terms. Thus, Eq. 13 can be reformulated as follows

$$\hat{\mathcal{G}}(x) + \hat{\mathcal{H}}(x) \leq \hat{\mathcal{F}}(c) \quad (14)$$

where $\hat{\mathcal{G}}(x) = \|x_{start} - x\|_2 * \frac{\mathcal{G}(c)}{\lambda_1}$ and $\hat{\mathcal{H}}(x) = \|x_{goal} - x\|_2 * \frac{\mathcal{G}(c)}{\lambda_1}$. Consequently, according to Eq. 12, we can derive that Eq. 14 satisfies

$$\|x_{start} - x\|_2 + \|x_{goal} - x\|_2 \leq \mu_1 + \lambda_1 < \hat{\mathcal{F}}(c) \quad (15)$$

where $\|\cdot\|_2$ denotes the two-norm of a vector. This is the general equation for an m -dimensional hyper-ellipsoid. The foci are x_{start} and x_{goal} , the major axis diameter is $\hat{\mathcal{F}}(c)$, and the minor axis diameters are

$$\sqrt{\hat{\mathcal{F}}(c)^2 - \|x_{start} - x_{goal}\|_2^2} \quad (16)$$

where $\|\cdot\|_2^2$ represents the square of $\|\cdot\|_2$. As shown in Fig. 3, in which c is the extended node, x is the next node to which to extend the tree and $\varepsilon = \|x_{start} - x_{goal}\|_2$ is the focal length. The shape of the hyper-ellipsoid is determined by the focal length ε and the estimated path length $\hat{\mathcal{F}}(c)$. From the above, we observe that the nodes that can optimize $\hat{\mathcal{F}}(c)$ are in $\zeta(x)$, the hyper-spheroid defined by Eq. 15, which indicates that sampling directly from this hyper-spheroid instead of from the whole set is an effective method, and this has been proven in [27].

Hereafter, the conclusions regarding this hyper-ellipsoid that are used in this paper are briefly described; the detailed proofs can be found in [27, 30–33].

Remark 1 When there are no obstacles, the convergence speed of this hyper-ellipsoid, η , is linear and is related only to m , the dimensionality of the workspace [30], such that.

$$\eta = \frac{m-1}{m+1} \quad (17)$$

as shown in Fig. 4.

Remark 2 A random sampling $x_{ellipse} \sim U(X_{ellipse})$ of an m -dimensional hyper-ellipsoid can be equivalently obtained by randomly sampling from the unit m -dimensional hypersphere $x_{sphere} \sim U(X_{sphere})$ [31–33].

3.2 Dynamic programming

Generally, for path planning problems, more resources will be needed to plan a path directly. Simultaneously, as mentioned above, the target of path planning is to continuously shorten the distance to the goal until either the node tree contains it or the distance to it is within one step. Fortunately, this coincides with the notion of dynamic programming. To improve the convergence time, appropriate nodes should be selected as new start nodes to progressively simplify the problem, thereby ensuring that the estimated cost to x_{goal} can be optimized as much as possible in each iteration.

Nevertheless, multiple candidates may exist in the node tree. To select the most appropriate one, Pareto dominance [34] is employed to prioritize the tree nodes. This concept is defined as follows.

Definition 4. (Pareto dominance): Given two vectors $u = [u_1, u_2, \dots, u_k]$ and $v = [v_1, v_2, \dots, v_k]$ such that $u, v \in \mathbb{R}^k$, if they satisfy

$$\forall i \in [1, k] \ v_i \leq u_i \text{ and } \exists j \in [1, k] \ v_j < u_j$$

Fig. 3 The informed subset that can optimize the estimated path, $\zeta(x)$, is an ellipse when planning in an $\mathbb{R}^2$ environment

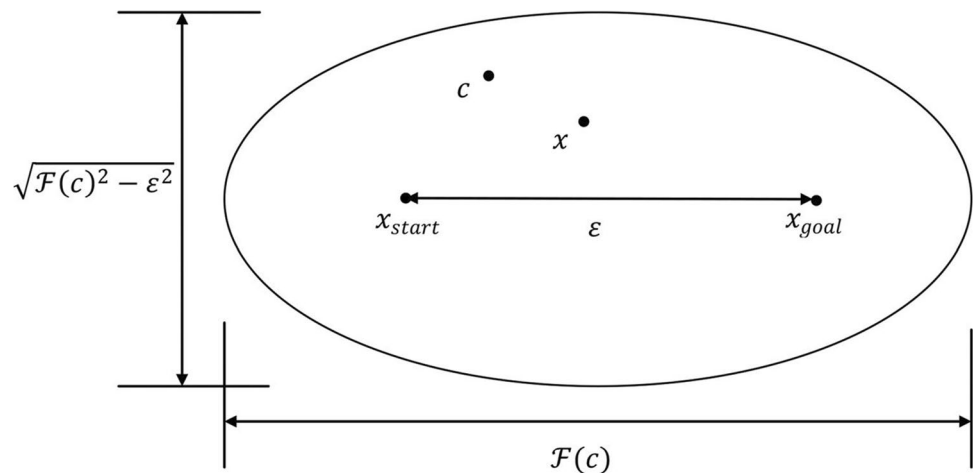
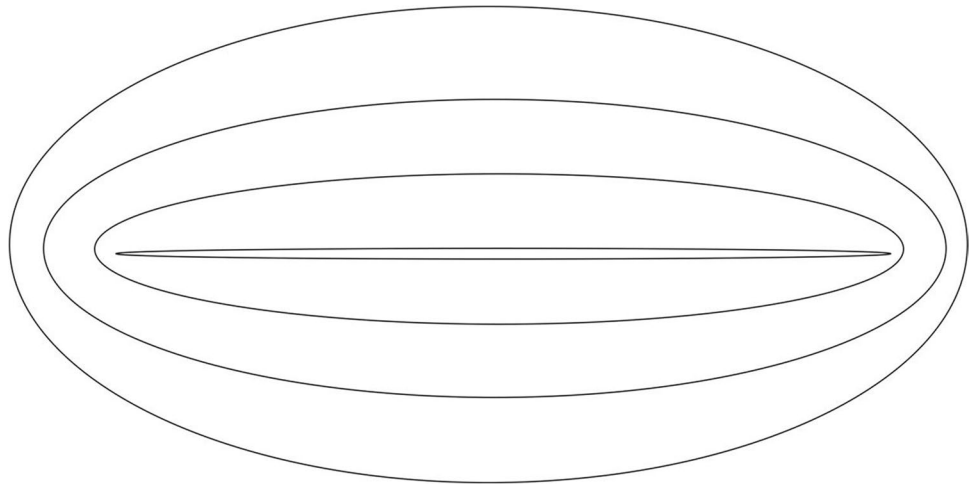


Fig. 4 A sequence of hyper-ellipsoids that have the same focal points, where each is obtained by direct sampling from the previous one when there are no obstacles; this sequence converges linearly to the ideal smallest



where k represents the dimensionality of each vector, then it is said that $\mathbf{v}$ dominates $\mathbf{u}$, denoted by $\mathbf{v} < \mathbf{u}$. Otherwise, they are non-dominated.

To construct the Pareto vector $\mathbf{v} \in \mathbb{R}^k$ of each tree node x , its out-degree $\mathcal{O}(x)$ and estimated cost $\hat{\mathcal{F}}(x)$ are employed. The out-degree, $\mathcal{O}(x) \in \mathbb{R}_{\geq 0}$, is defined as the number of child nodes of x that are directly connected to it in the topology diagram, and $\mathcal{O}(x) = 0$ denotes that there is no child node connect to x . As illustrated in Fig. 5, $x_3(3)$ designates that x_3 has three child nodes, x_4 , x_6 and x_7 , and $x_6(0)$ signifies that x_6 is a leaf node without child nodes. Considering the RRT planning problem, if $\mathcal{O}(x) > 0$, this means that the cost of a path from x_{start} to a child of x that is constrained to pass through x is smaller than that of other paths. This also means

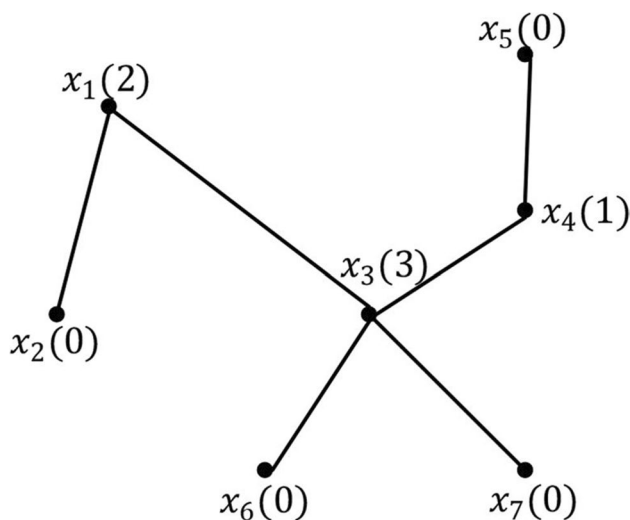


Fig. 5 Topology diagram of a node tree, with the $\mathcal{O}(x)$ values in parentheses

that the corresponding $\mathcal{G}(x)$ is smaller than others. Moreover, the higher $\mathcal{O}(x)$ is, the higher the probability that taking node x as the next start node may lead to optimization of the path.

In summary, it can be observed $\mathcal{F}(x)$ should be as small as possible, while $\mathcal{O}(x)$ should be as large as possible. Thus, the Pareto vector $\mathbf{v}$ can finally be constructed as

$$\mathbf{v} = [-\mathcal{O}(x), \mathcal{F}(x)]$$

As illustrated in Fig. 6, a node tree containing several nodes is expanded into a topology diagram. In Fig. 6b, each node $\mathbf{v}_i$ represents the Pareto vector of node x_i , and an edge between two vectors represents a domination relationship. If one vector dominates another, the edge between them points to the one that is dominated. For example, the edge between $\mathbf{v}_3$ and $\mathbf{v}_6$ indicates that $\mathbf{v}_3$ dominates $\mathbf{v}_6$. If there is no edge between two nodes, this means there is no domination relationship between them.

For a Pareto vector $\mathbf{v}$, the out-degree $\mathbf{v}$ is redefined as the number of vectors it dominates. In Fig. 6c, $\mathbf{v}_3$ represents that $\mathbf{v}_3$ dominates three vectors, $\mathbf{v}_1$, $\mathbf{v}_2$, and $\mathbf{v}_6$, as shown in Fig. 6b. A vector with a maximal $\mathbf{v}$ is a Pareto-optimal vector, and the corresponding node is a Pareto-optimal node. Here, we can observe that $\mathbf{v}_4$ is the optimal vector, with $\mathbf{v}_4$. Therefore, x_4 is the Pareto-optimal node and will be selected as the new start node. Generally, there will often be more than one Pareto-optimal vector. We demonstrate such a situation by removing node x_5 from Fig. 6, as shown in Fig. 7. We can observe that $\mathbf{v}_4$ no longer dominates $\mathbf{v}_5$, causing its out-degree to decrease to 3. Consequently, it has the same out-degree as $\mathbf{v}_3$. In such a case, the new start node will be randomly selected from among the Pareto-optimal vectors;

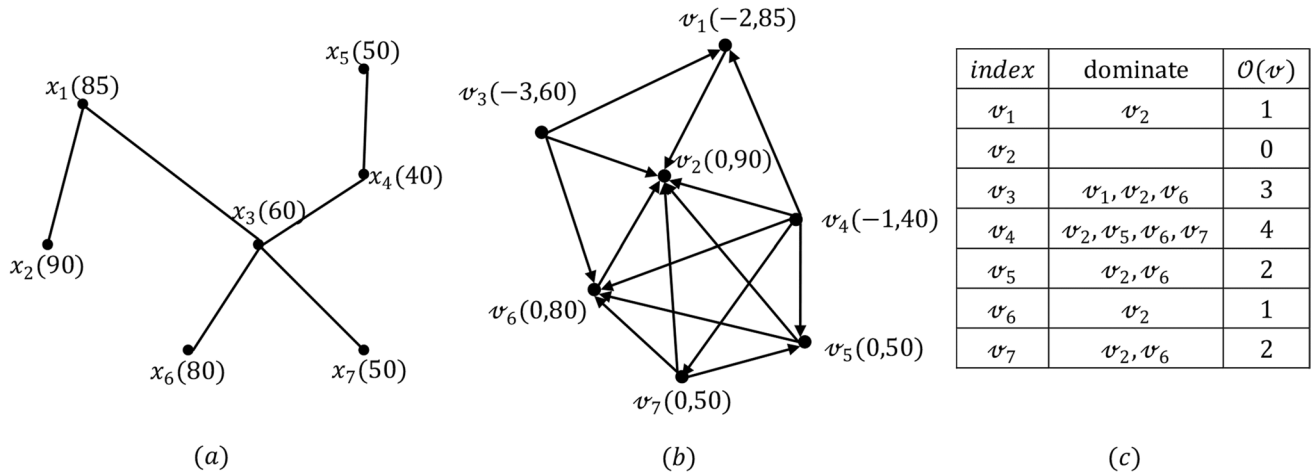


Fig. 6 **a** is the topology diagram of a node tree, $x_i \in \mathbb{R}(i \in [1, 7], i \in \mathbb{N})$ is the i -th node, and the value in parentheses denotes the estimated cost $\mathcal{F}(x_i)$ of the node x_i ; **b** is the topology diagram of Pareto vector of the node in (a), v_i is the node that

represents the Pareto vector of node x_i , and the value in parentheses denotes the specific value of Pareto vector v_i ; **c** The second column shows the vectors dominated by the vector in the first column, and the third column gives the number of the it dominates, $\mathcal{O}(v_i)$

here, either x_3 or x_4 will be randomly chosen as the new start node.

However, given the basis of the Pareto principle that approximately 80% of wealth is concentrated in approximately 20% of a population [35], we can infer that the node

selected based on Pareto dominance will not always be the best. Consequently, we also should instead randomly select a node from among the nonoptimal ones with a certain probability to avoid becoming trapped in a local optimum.

The pseudocode is given in Algorithm 2.

Algorithm 2 Selection of a Pareto-optimal node

Input: V : the extended node tree;

$\hat{F}(x)$: the estimated path length of each node x in the node tree;

x_{goal} : the end node of planning problem;

p : the probability of randomly selecting a node as Pareto optimal node;

Output: x_{pareto} : the Pareto optimal node selected through Pareto dominance;

1. Generate rp randomly, $rp \in (0,1)$
2. $x_{pareto} \leftarrow \emptyset$
3. **for** $\forall x_i$ in V
4. $v_i \leftarrow (\mathcal{O}(x_i), \hat{F}(x_i))$
5. $v \leftarrow v \cup v_i$
6. **end for**
7. **for** $\forall v_p, v_q$ in v
8. **if** $\forall i \in \mathbb{N}: 1 \sim m \ v_p[i] \leq v_q[i]$ and $\exists j \in \mathbb{N}: 1 \sim m \ v_p[j] < v_q[j]$
9. $\mathcal{O}(v_p)++$
10. **end if**
11. **end for**
12. $sort(V, v, count)$
13. $x_{pareto} = choose(V)$
14. **if** $rp < p$
15. $x_{pareto} = Nearest(V, x_{goal})$
16. **end if**

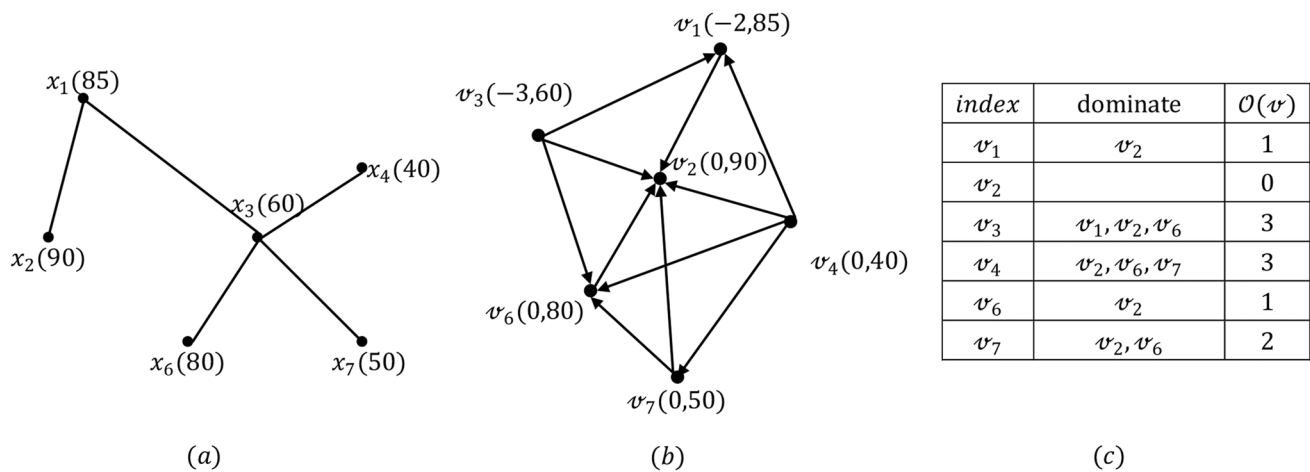


Fig. 7 Corresponding situation in Fig. 3 after the removal of x_5

In the algorithm above, the function $\mathbf{v}$ sorts V and $\mathbf{v}$ in accordance with their values of $\mathbf{v}$, and the function $choose(V)$ randomly returns one Pareto-optimal node from among the nodes with the largest $\mathbf{v}$.

Once a new start node has been selected, we can ensure that its distance from the previous one is locally optimal. This process is repeated until the goal is reached, and accordingly, we can deduce that both the convergence time and path length are correlated with the interval of the Pareto dominance procedure. Thus, by selecting an appropriate interval, the algorithm can achieve a suitable balance between convergence time and path length.

3.3 Dynamic RRT

The proposed algorithm, which samples new nodes directly from the informed subset and selects new start nodes based on Pareto dominance, is named Dynamic RRT. Its pseudocode is shown in Algorithm 3. It is similar in structure to RRT, except for the additional lines marked

in red in Algorithm 3. Different from RRT and Informed RRT*, Dynamic RRT aims to generate a path while balancing convergence time and path length, without computing an initial path to be refined.

The proposed algorithm is different from these existing methods in that once a new node is added to the node tree, the node x_c is confirmed by selecting the nearest node to x_{goal} . Then, the estimated cost of a path from x_{start} to x_{goal} constrained to pass through x_c , $\hat{\mathcal{F}}(c)$, is calculated and used together with the focal length to determine the shape of the hyper-ellipsoid that is applied to restrict the informed subset. Thus, the proposed algorithm focuses on finding a new node by sampling directly from this hyper-ellipsoid, which may optimize the estimated path cost, as shown in Lines 5 and 6 of Algorithm 3. Moreover, a counter is maintained that increases with the number of iterations. When this counter reaches a preset interval and $CheckCollision(x_{pareto}, x_{pareto})$ returns false, the $ChooseNodePareto$ procedure will go into effect to select a Pareto-optimal node as a new start node, as shown in Lines 14–21 of Algorithm 3.

Algorithm 3 *DynamicRRT*

Input: x_{start} : the start node of the planning problem;
 x_{goal} : the end node of the planning problem;
 X : the total workspace;

Output: V : the node tree generated by the algorithm;
 $parent$: the parent node of each tree node;

1. $V \leftarrow x_{start}$
2. $x_c \leftarrow x_{start}$
3. $parent = 1$
4. **while** $ReachGoal = false$
5. $\mathcal{F}(c) = CalCostHat(x_{start}, x_{goal}, x_c, Cost(x_{start}, x_c))$
6. $x_{rand} = SampleEllipsoid(x_{start}, x_{goal}, X, \mathcal{F}(c))$
7. $x_{nearest} = Nearest(V, x_{rand})$
8. $x_{new} = Steer(x_{nearest}, x_{rand}, step)$
9. **if** $CheckCollision(x_{nearest}, x_{new}) = false$
10. $parent(x_{new}) = x_{nearest}$
11. $V = V \cup x_{new}$
12. $CostNew = Cost(x_{start}, x_{nearest}) + lineDistance(x_{nearest}, x_{new})$
13. $Cost = Cost \cup CostNew$
14. $x_c = Nearest(V, x_{goal})$
15. $counter = counter + 1$
16. **if** $counter = interval$
17. $x_{pareto} = ChooseNodePareto(V, x_{goal})$
18. **if** $CheckCollision(x_{pareto}, x_{pareto})$
19. $x_{start} = x_{pareto}$
20. **end if**
21. **end if**
22. **end if**
23. **end while**

4 Simulation

This section reports several simulations performed to verify the performance of the Dynamic RRT algorithm. Initially, it is confirmed that Dynamic RRT achieves a balance between convergence time and path length by comparing its performance for distinct intervals of the Pareto dominance procedure. Then, Dynamic RRT is compared with several traditional and advanced RRT variants in environments with randomly generated obstacles, verifying its stability and robustness in different environments. The specific simulation environment is MATLAB 2019a, and the PC is configured with a 2.9 GHz AMD Ryzen R7-4800H CPU and 16 GB of RAM.

4.1 Performance analysis of Dynamic RRT

4.1.1 Configuration information

Simulations were conducted in different dimensions and with different intervals to evaluate the performance of Dynamic RRT in balancing the convergence time and path length, while the environment size and obstacle density remained unchanged (Table 1). The convergence time and path length are considered as the indicators, where the convergence time is the time needed to converge to a solution and the path length is considered equivalent to the path cost, which is the cost of the path from the start node to the goal node.

Table 1 Environment configurations

dimension	environment	obstacles	start	goal
2	1500*1500	225	(400, 400)	(1100, 1100)
3	1500*1500*1500	400	(1, 1, 1)	(1100, 1100, 1100)

4.1.2 Simulation results

A. Interval can affect the performance As shown in Fig. 8, we can see that in either two or three dimensions, the length of the path generated by Dynamic RRT improves as the interval of the Pareto dominance procedure decreases. Additionally, the convergence time monotonically decreases as the interval decreases within the range of (20,4) in two dimensions (Table 2). Nevertheless, as shown in Fig. 8a and e, a smaller interval cannot always lead to a better path when the interval is smaller than a certain limit. On the contrary, the paths become unsmooth compared with (b) and (f). Moreover, from Table 2, we observe that the time consumed in these cases has risen compared with the others. This is because when the interval takes an excessively small value, such as 3, the number of nodes exhibiting Pareto dominance decreases to 2, such that the Pareto-optimal node is randomly selected between only these two nodes, leading to an inability to take full advantage of Pareto dominance. The same problem also arises as the interval becomes larger. Figure 8d and h illustrate the results of simulations with an interval of 20, from which we can again observe the phenomenon that the convergence time and path cost rise, as in the case of an excessively small interval. As the interval tends toward infinity, the Pareto dominance mechanism may even fail entirely, causing Dynamic RRT to degenerate into RRT with direct sampling from the informed subset. Accordingly, the interval needs to be selected within a reasonable range.

B. Influence of different intervals After that, we simulated the influence on the path cost and convergence time of different intervals within the range of (2,200). The results are illustrated in Fig. 9. From Fig. 9a, on the one hand, we can see that when the interval exceeds an upper bound of 80, the convergence time hardly rises, indicating that Dynamic RRT has degenerated into RRT with direct sampling from the informed subset, although the path cost still fluctuates randomly. On the other hand, we can observe that when the interval is less than a lower bound of 4, the convergence time begins to rise, while the path cost still maintains a downward trend. This implies that by continuing to decrease the interval below 4, we can still obtain a better path, although the situation depicted in Fig. 8a and e may appear. Between these upper and lower bounds, downward

trends are maintained in both the convergence time and the path cost. Consequently, we can achieve a balance between convergence time and path length by selecting an appropriate interval in this range. Under this condition, an interval of 4 will result in a convergence time of 0.034 s and a path cost of 1157.4. If a shorter path is needed, the interval can be set to 2 or 3, at the cost of a significant increase in the convergence time.

4.2 Comparison with different RRT variants

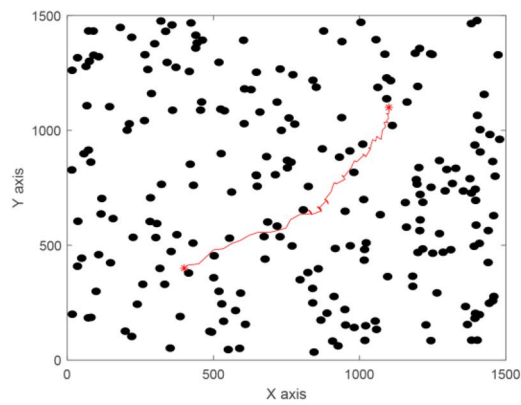
4.2.1 Configuration information

Several traditional (RRT, RRT*) and advanced (Informed RRT*, LBT-RRT) RRT variants are selected for comparison to verify the performance and stability of Dynamic RRT in balancing the convergence time and path length. For illustration, the 2-dimensional environment specified in Table 1 is taken as an example. The convergence time and path length are still used as the indicators; however, the convergence time of Informed RRT* has a different meaning from the others, defined as the time spent for Informed RRT* to converge to a path of the same length as Dynamic RRT. Each algorithm was given 10 s to generate a path. We performed 100 runs under each condition.

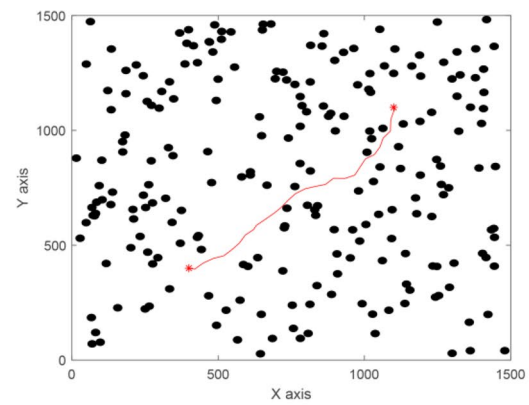
4.2.2 Performance comparison

Table 3 compares the performance of Dynamic RRT with the traditional variants of RRT. From Table 3, it can be seen that RRT* is slower than RRT while achieving higher quality, and both of their indicators improve when the probability of new node selection is biased toward the goal. In addition, it is obvious that with an interval of 8, Dynamic RRT generates a path with a better convergence time than RRT* while maintaining the same path length. With an interval of 6, the path generated by Dynamic RRT is better than that generated by RRT in terms of path length while consuming the same time. Furthermore, when the interval is 4, Dynamic RRT achieves not only a shorter convergence time than RRT but also a shorter path length than RRT*, thus clearly showing its superiority in balancing the convergence time and path length compared with the traditional RRT variants.

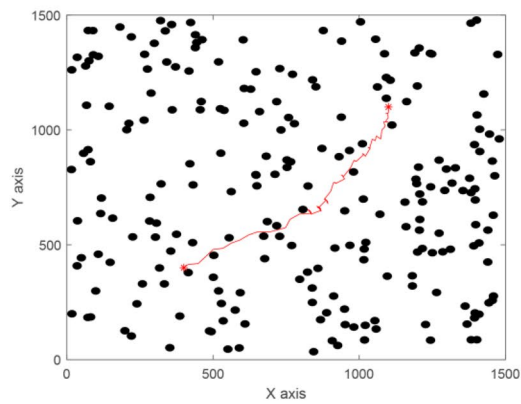
In addition to these traditional RRT variants, advanced variants are also considered to further demonstrate the performance of the proposed algorithm. From Table 4, it can be observed that Informed RRT*, an improved version of RRT*, requires more time and iterations to reach the same path length as Dynamic RRT. Although a shorter path can be obtained by Informed RRT* when no limit is placed on the run time, this is not significantly useful because of the extreme increase in the convergence time.



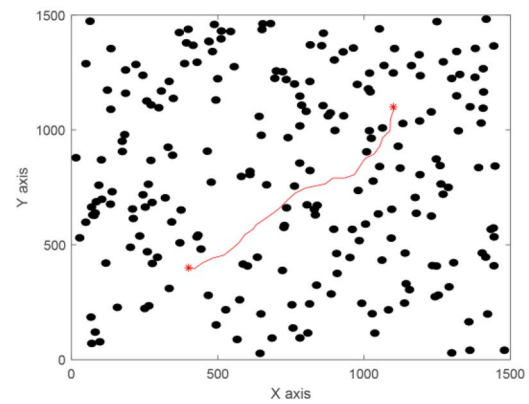
(a)



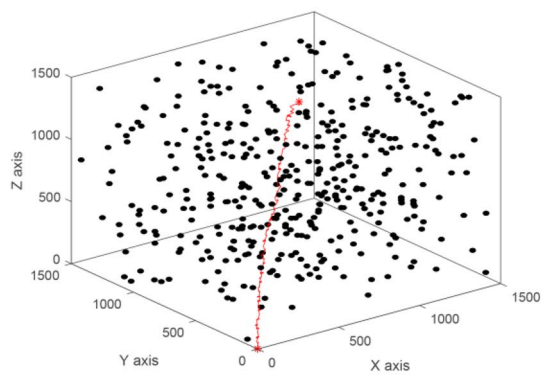
(b)



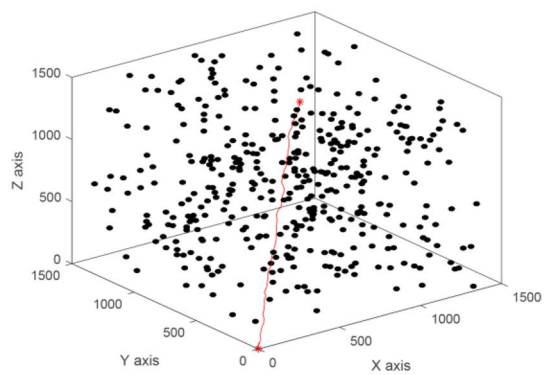
(c)



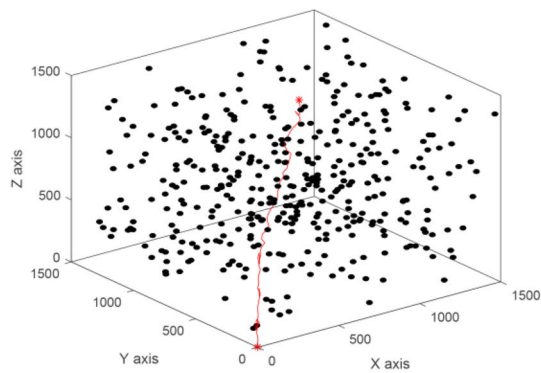
(d)



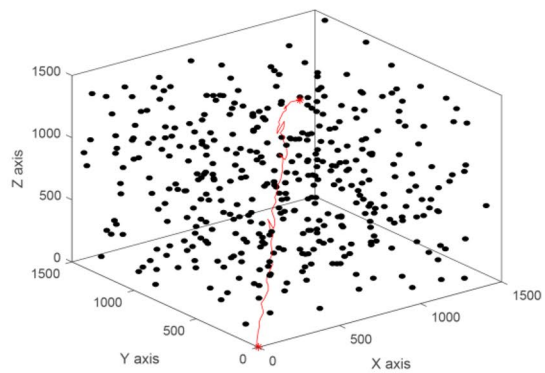
(e)



(f)



(g)



(h)

Fig. 8 Results of path planning in different dimensions and with different intervals. The black m-balls with a radius of 15 represent uniformly distributed obstacles. The start and goal nodes are set as shown in Table 1. Figure 8a-d and e-h illustrate the performance of Dynamic RRT in 2-dimension and 3-dimension with intervals of 3, 4, 10, and 20

Table 2 Results of Dynamic RRT with different intervals in $\mathbb{R}^2$, corresponding to Fig. 8a-d; the time and length are average values over 100 trials

Interval	Convergence Time	Path Length
3	0.195 s	1136.0
4	0.034 s	1157.4
10	0.072 s	1197.5
20	0.108 s	1250.7

In addition, for the case in which RRT and RRT* are taken as the indicators, article [28] demonstrates that the path generated by LBT-RRT is shorter in path length than RRT and less in convergence time than RRT*, which means LBT-RRT is higher in convergence time than RRT and longer in path length than RRT*, as is shown in Table 5. From the above analysis, it could be deduced that Dynamic RRT is not only less in convergence time, but also shorter in path length than LBT-RRT. That is to say, the performance of Dynamic RRT is better than that of LBT-RRT.

4.2.3 Stability comparison

Another set of simulations was executed to verify the stability of Dynamic RRT. Three different environmental ranges

Table 3 Results of each algorithm; the parameters are average values over 100 runs, the convergence time is defined as the time spent to converge to a solution for the first time, and the dash line indicates that the parameter is not applicable for the algorithm

Algorithm	Interval	Bias	Convergence time(s)	Length	Iterations
RRT	-	0%	0.582	1342.54	1101.7
	-	10%	0.059	1274.69	282.6
	-	20%	0.045	1234.56	227.1
RRT*	-	0%	1.432	1309.50	1041.1
	-	10%	0.123	1200.28	160.0
	-	20%	0.099	1186.27	142.6
Dynamic RRT	4	-	0.034 s	1157.42	91.8
	6	-	0.042 s	1175.41	132.7
	8	-	0.072 s	1183.58	164.3

with the same obstacle density were considered: from small to large, 1000*1000, 1500*1500, and 2000*2000. The distance between the start and goal nodes was kept constant by adjusting them along with the environment. The detailed configurations are shown in Table 6. Similarly, RRT, RRT*, and Informed RRT* are selected for the comparison. Figure 10 illustrates the distribution of paths generated by different algorithm in env2. It can be concluded that Dynamic RRT is more stable than the previous RRT variants in this typical environment.

For generalization, the algorithms are performed 100 times in the three different environments in Table 6. Figure 11 shows one set of results for env1, and the details are given in Table 7. From them, it can be observed that

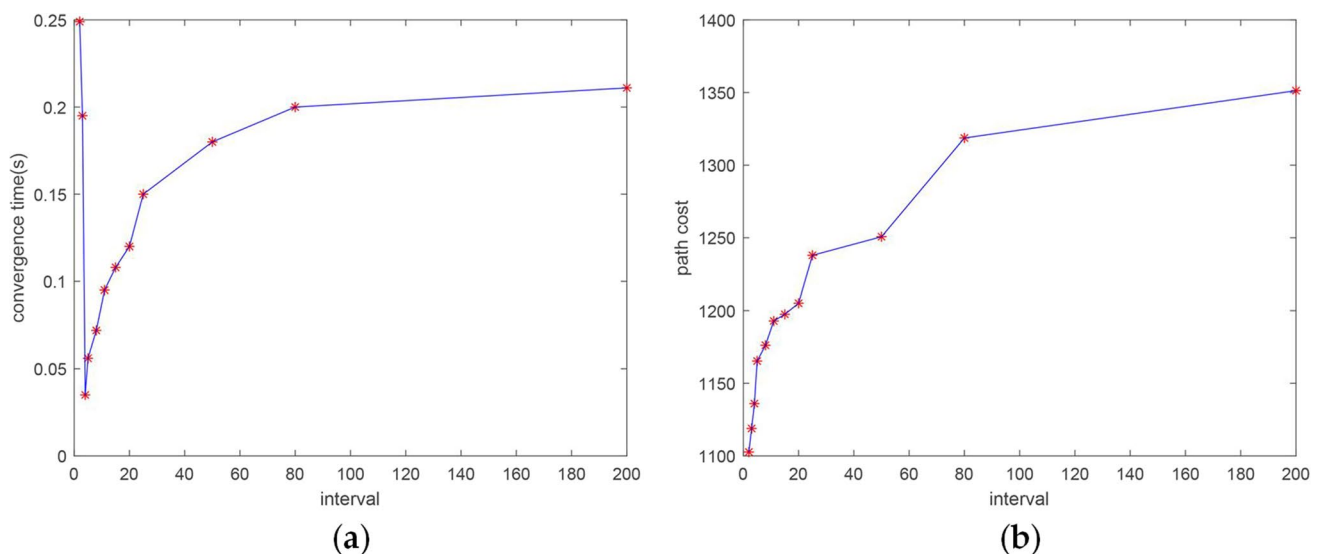


Fig. 9 Illustration of the influence of different intervals on the path cost and convergence time. We ran ten simulations in each interval: (a) influence on convergence time; (b) influence on path cost

Table 4 Results of Dynamic RRT with setting the interval as 4 and Informed RRT* with setting maximum iterations as 3000. The parameters are average values over 100 runs. Specially, the convergence time of Informed RRT* is defined as the time spent to converge to a path of the same length as Dynamic RRT

Algorithm	Convergence time	Path length	Iterations
Dynamic RRT	0.034 s	1157.42	91.8
Informed RRT*	2.849 s	1168.05	1225.3

the convergence time and path length of Dynamic RRT are approximately the same in any environment, which can be attributed to the direct sampling from an informed subset that depends only on the estimated cost. In contrast, the RRT and RRT* algorithms explore the entire environment evenly, and consequently, the time to find a solution

Table 5 Results of Dynamic RRT and LBT-RRT

Algorithm	Convergence time	Path length
Dynamic RRT	< RRT	> RRT*
LBT-RRT	< RRT*	> RRT

increases significantly with the problem range. Although we can alleviate this problem by setting a certain probability of bias toward the goal, it still exists.

Table 8 shows the comparison with Informed RRT*, in which an informed subset is also employed. From Table 8, it could be noticed that Informed RRT* spends more time to converge to a path of the same length as Dynamic RRT in all three environments. Moreover, its convergence time increases rapidly as the size of the environment increases.

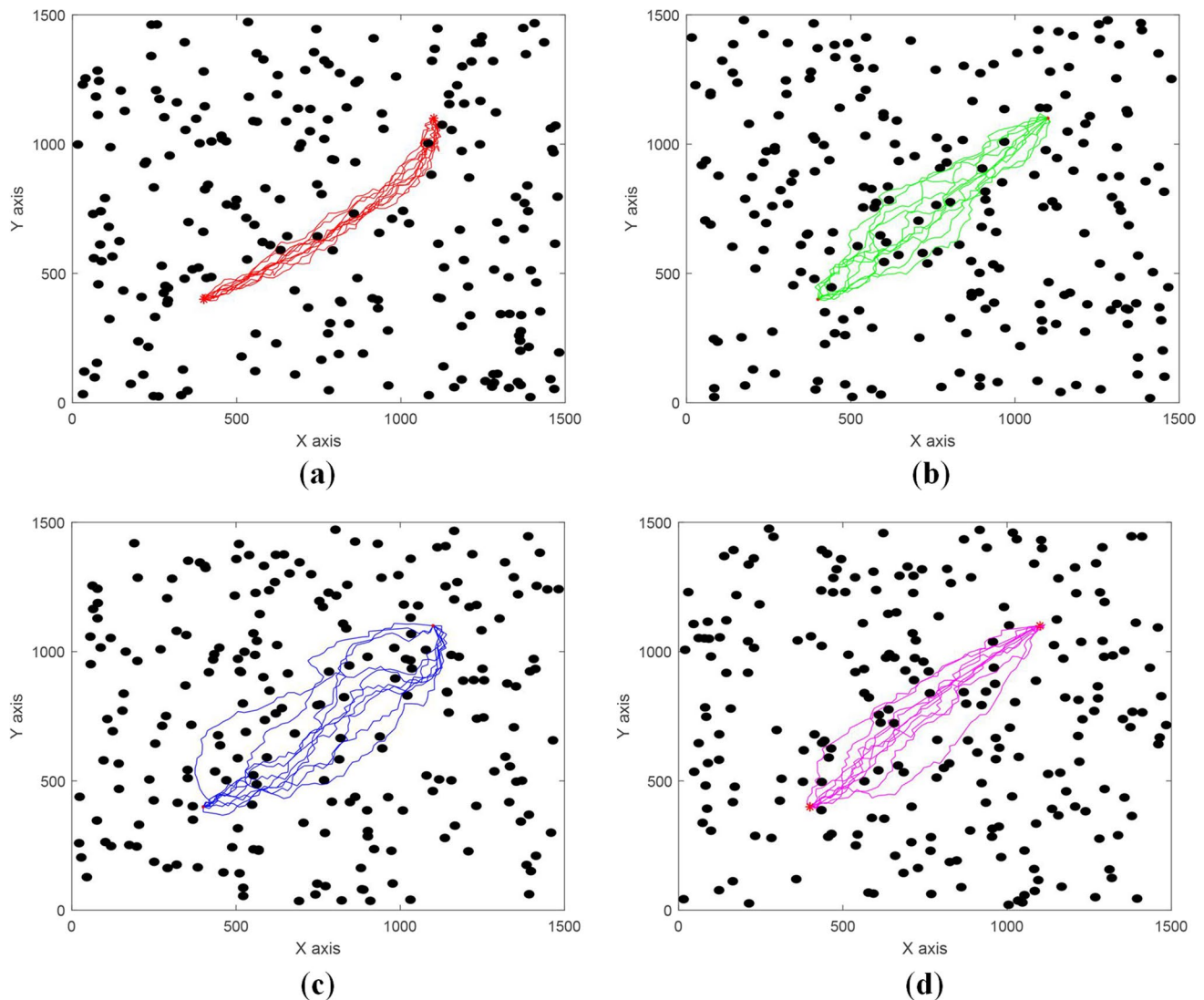


Fig. 10 Generated paths in env2 over ten runs: (a) Dynamic RRT with an interval of 4; (b) RRT* with a bias toward the goal of 10%; (c) RRT with a bias toward the goal of 10%; (d) Informed RRT*

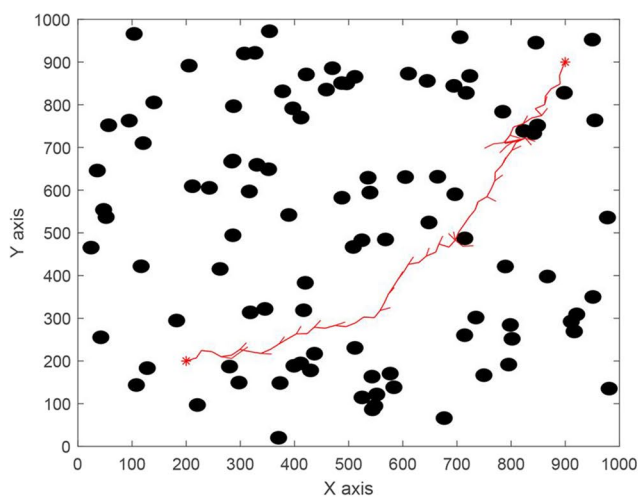
Table 6 Detailed configurations

Environment	Range	Obstacles	Start	Goal
env1	1000*1000	100	(200, 200)	(900, 900)
env2	1500*1500	225	(400, 400)	(1100, 1100)
env3	2000*2000	400	(700, 700)	(1400, 1400)

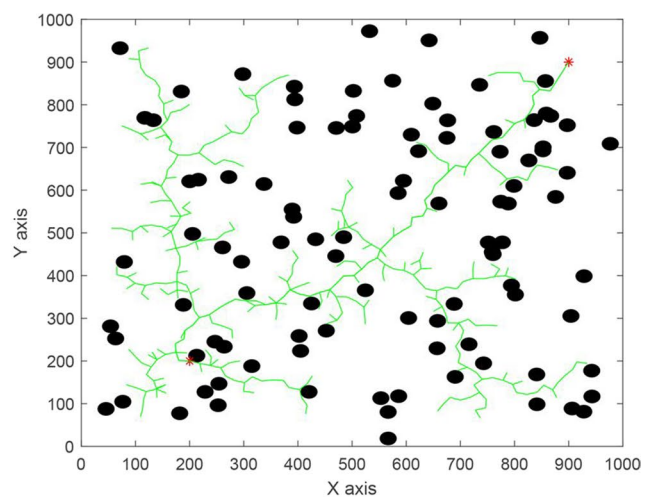
That is because, the problem is decomposed into subproblems on the basis of Pareto dominance by selecting Pareto-optimal nodes as new start nodes, which further reduces the sensitivity of the proposed algorithm the range of the problem. In contrast, Informed RRT* strongly depends on the hyper-ellipsoid determined by the initial path length. And it does not activate its informed subset until the underlying

Table 7 Results of each algorithm in different environmental ranges, where the values of the indicators are averaged over 100 runs and the convergence time is defined as the time spent to converge to a solution for the first time

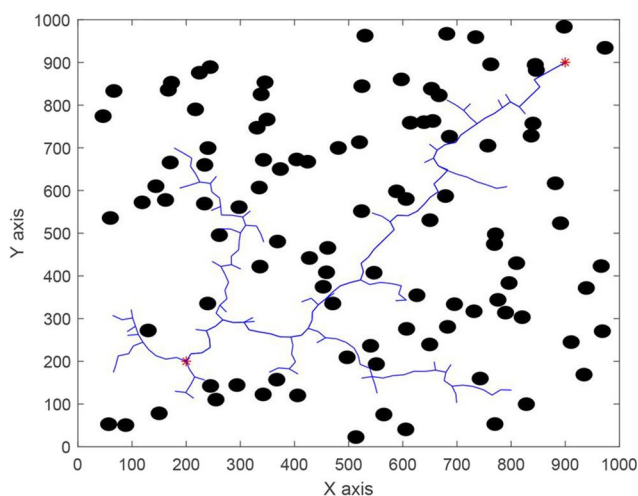
Algorithm	Environment	Convergence time	Path length
RRT	env1	0.112 s	1309.84
	env2	0.582 s	1365.17
	env3	1.698 s	1426.66
RRT*	env1	0.239 s	1269.86
	env2	0.848 s	1309.50
	env3	1.951 s	1340.51
Dynamic RRT	env1	0.021 s	1158.82
	env2	0.034 s	1157.42
	env3	0.067 s	1163.25



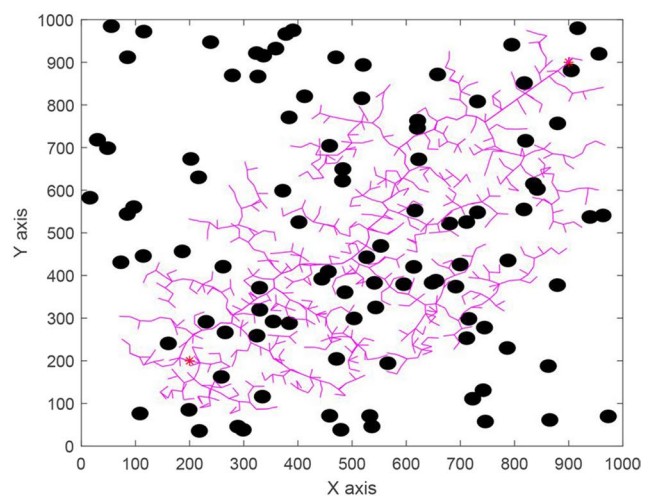
(a)



(b)



(c)



(d)

Fig. 11 Extended node trees in env1, with red stars representing the start and goal nodes: (a) Dynamic RRT with an interval of 4; (b) RRT* with a bias toward the goal of 10%; (c) RRT with a bias toward the goal of 10%; (d) Informed RRT*

Table 8 Results of Dynamic RRT and Informed RRT* in different environment. The success rate refers to whether Informed RRT* converges to a path of the same length as Dynamic RRT in a finite number of iteration and finite time. The convergence time of Informed

RRT* is defined as the time spent to converges to a path of the same length as Dynamic RRT, and the one of Dynamic RRT is just the time spent to converge to the solution

Algorithm	Environment	Success rate	Convergence time	Path length
Dynamic RRT	env1	-	0.021 s	1158.82
	env2	-	0.034 s	1157.42
	env3	-	0.067 s	1163.25
Informed RRT*	env1	62.5	1.1677 s	1164.94
	env2	60.1	2.849 s	1162.77
	env3	55.6	5.417 s	1168.60

RRT* algorithm has computed its initial solution, which means taking more time to generate the initial path with the environment increasing. Figure 12 shows the phenomenon. It is obvious that Dynamic RRT not only can generate a better path in a very short time but also offers strong stability against variations in the distance between the start and goal nodes, which Informed RRT* does not.

4.3 Discussion

The aim of this paper is to generate a path balancing the convergence time and path length. From the results obtained from the experiments, the following observations can be made:

- 1) The performance analysis demonstrates the ability to balance the considerations of convergence time and path length by adjusting the interval of the Pareto dominance procedure. It also shows that even better paths can be generated at the expense of a longer convergence time.
- 2) The performance comparison simulations compared with traditional RRT variants demonstrate that its convergence time is shorter than that of RRT, and path

length is better than that of RRT*. Furthermore, compared with LBT-RRT, it performs better in balancing the convergence time and path length. Even, it could converge to a better path without generating the initial one while Informed RRT* takes more time to compute an equivalent path.

- 3) The stability comparison simulations demonstrate that Dynamic RRT is more stable than the variants mentioned above under various environments and varying distances between the start and goal nodes. In addition, the paths it generates are more concentrated.

5 Conclusion

This paper introduced a Dynamic RRT algorithm for balancing the convergence time and path length in path planning. It improves the new node generation efficiency and quality by constraining to sample directly from an informed subset and decomposing the problem into subproblems. The simulation results show that convergence time and path length can be traded-off by designating the interval of the Pareto dominance procedure, and Dynamic RRT performs better than given several traditional and advanced RRT variants. This means a better path can be generated by this algorithm within the same time or less time as other algorithms. It will save a lot of time for other computing in the application of robots and the strong stability and robustness also make it possible to apply in industrial production.

In future work, an interesting challenge will be to use our method to fit any cost function, as RRT* does, rather than optimizing only the Euclidean distance. It will also be interesting to apply our method to various robots, whether a robot car or a high-degree-of-freedom manipulator, to investigate its superiority in various dimensions.

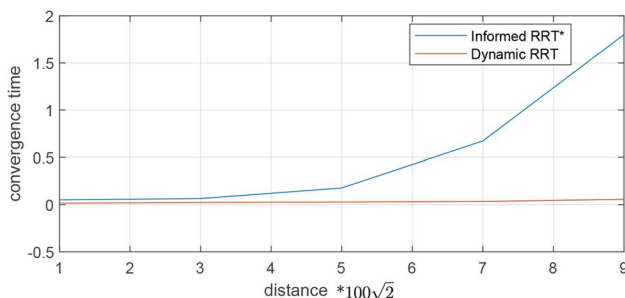


Fig. 12 Convergence time vs. distance

Authors' Contributions All authors contributed to the study conception and design. Penglei Zhao wrote the manuscript and did the research. Technical support was provided by Yinghui Chang, Weikang Wu, Hongyin Luo, Zhixin Zhou. Valuable comments on manuscript revisions were put forward by Yanping Qiao, Ying Li, Chenhui Zhao, Zenan Huang, Bijing Liu, Xiaojie Liu, Shan He. Prof. Donghui Guo provided manuscript writing guidance. All authors read and approved the final manuscript.

Funding This work is supported by the Key project's funding of NSFC (No.61836010).

Code availability The source code in this paper will be disclosed upon acceptance for publication of this paper. Before accepting, please E-mail to 2,399,496,491@qq.com if the source code is needed.

Declarations

Ethics approval Not applicable.

Competing interests The authors have no relevant financial or non-financial interests to disclose.

Consent to participate Not applicable.

Consent for publication Not applicable.

References

- Ji, S., et al.: Learning-Based Automation of Robotic Assembly for Smart Manufacturing. *Proc. of the IEEE*. **109**, 423–440 (2021)
- Perzylo, A., et al.: SMErobotics Smart Robots for Flexible Manufacturing. *IEEE ROBOT AUTOM MAG*. **26**, 78–90 (2019)
- Hvilshoj, M., et al.: Autonomous industrial mobile manipulation (AIMM): past, present and future. *Industrial Robot-the International Journal of Robotics Research and Application*. **39**, 120–135 (2012)
- Roa, M.A., Berenson, D., Huang, W.: Mobile Manipulation: Toward Smart Manufacturing. *IEEE Robot. Autom. Mag.* **22**, 14–15 (2015)
- Gonzalez, A.G.C., et al.: Supervisory Control-Based Navigation Architecture: A New Framework for Autonomous Robots in Industry 4.0 Environments. *IEEE Trans. Indust. Inf.* **14**, 1732–1743 (2018)
- Pan, C., et al.: A Novel Algorithm for Wafer Sojourn Time Analysis of Single-Arm Cluster Tools With Wafer Residency Time Constraints and Activity Time Variation. *IEEE Transactions on Systems Man Cybernetics-Systems*. **45**, 805–818 (2015)
- Li, Z., et al.: A Fault-Tolerant Method for Motion Planning of Industrial Redundant Manipulator. *IEEE Trans. Industr. Inf.* **16**, 7469–7478 (2020)
- Baumann, D., et al.: Wireless Control for Smart Manufacturing: Recent Approaches and Open Challenges. *Proc. IEEE* **109**, 441–467 (2021)
- Li, S., Han, K., Li, X., et al.: Hybrid Trajectory Replanning-Based Dynamic Obstacle Avoidance for Physical Human-Robot Interaction. *J Intell Robot Syst* **103**, 41 (2021)
- Hart, P.E., Nilsson, N.J., Raphael, B.: A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*. **4**, 100–107 (1968)
- Karaman, S. and E. Frazzoli.: Incremental Sampling-based Algorithms for Optimal Motion Planning. in *Robotics: Sci. Syst.* **2010**. (2010). <https://doi.org/10.48550/arXiv.1005.0416>
- Chiang, H.T.L., et al.: RL-RRT: Kinodynamic Motion Planning via Learning Reachability Estimators From RL Policies. *IEEE Robotics and Automation Letters*. **4**, 4298–4305 (2019)
- Wang, J.K., et al.: Neural RRT*: Learning-Based Optimal Path Planning. *IEEE Trans. Autom. Sci. Eng.* **17**, 1748–1758 (2020)
- Li, Y., et al.: Neural Network Approximation Based Near-Optimal Motion Planning With Kinodynamic Constraints Using RRT. *IEEE Trans. Industr. Electron.* **65**, 8718–8729 (2018)
- Lavalle, S.M.: Rapidly-exploring random trees : A new tool for path planning. *Research Report*. https://www.researchgate.net/publication/2639200_Rapidly-Exploring_Random_Trees_A_New_Tool_for_Path_Planning. (1998)
- Kavraki, L.E., et al.: Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE Trans. Robot. Autom.* **12**, 566–580 (1996)
- Wang, J.K., Meng, M.Q.H., Khatib, O.: EB-RRT: Optimal Motion Planning for Mobile Robots. *IEEE Trans. Autom. Sci. Eng.* **17**, 2063–2073 (2020)
- Kusuma, M., Riyanto, and C. Machbub.: Humanoid Robot Path Planning and Rerouting Using A-Star Search Algorithm. 2019 IEEE International Conference on Signals and Systems (Icsigsys). (2019). <https://doi.org/10.1109/ICSIGSYS.2019.8811093>
- An, B., Kim, J., Park, F.C.: An Adaptive Stepsize RRT Planning Algorithm for Open-Chain Robots. *IEEE Robotics and Automation Letters*. **3**, 312–319 (2018)
- Wang, W., Zuo, L., Xu, X.: A Learning-based Multi-RRT Approach for Robot Path Planning in Narrow Passages. *J. Intell. Rob. Syst.* **90**, 81–100 (2018)
- Aguinaga, I., Borro, D., Matey, L.: Parallel RRT-based path planning for selective disassembly planning. *Int. J. Adv. Manuf. Technol.* **36**, 1221–1233 (2008)
- Bruce, J. and M.M. Veloso.: Real-time randomized path planning for robot navigation. *Robocup 2002: Robot Soccer World Cup Vi.* 2752, 288–295 (2003)
- Jr, J. and S.M. Lavalle.: RRT-Connect: An Efficient Approach to Single-Query Path Planning. in *Proceedings of the 2000 IEEE International Conference on Robotics and Automation, ICRA 2000, April 24–28, 2000, San Francisco, CA, USA*. (2000).
- Moon, C.B., Chung, W.: Kinodynamic Planner Dual-Tree RRT (DT-RRT) for Two-Wheeled Mobile Robots Using the Rapidly Exploring Random Tree. *IEEE Trans. Industr. Electron.* **62**, 1080–1090 (2015)
- Karaman, S., Frazzoli, E.: Sampling-based algorithms for optimal motion planning. *Int. J. Robot. Res.* **30**, 846–894 (2011)
- Chen, L., et al.: A Fast and Efficient Double-Tree RRT*-Like Sampling-Based Planner Applying on Mobile Robotic Systems. *IEEE-Asme Transactions on Mechatronics*. **23**, 2568–2578 (2018)
- Gammell, J.D., S.S. Srinivasa, and T.D. Barfoot.: Informed RRT*: Optimal Sampling-based Path Planning Focused via Direct Sampling of an Admissible Ellipsoidal Heuristic. 2014 IEEE/Rsj International Conference on Intelligent Robots and Systems (Iros 2014). 2997–3004 (2014)
- Salzman, O., Halperin, D.: Asymptotically Near-Optimal RRT for Fast, High-Quality Motion Planning. *IEEE Trans. Rob.* **32**, 473–483 (2016)
- Qi, J., Yang, H., Sun, H.X.: MOD-RRT*: A Sampling-Based Algorithm for Robot Path Planning in Dynamic Environment. *IEEE Trans. Industr. Electron.* **68**, 7244–7251 (2021)
- Gammell, J.D., S.S. Srinivasa, and T.D. Barfoot.: On Recursive Random Prolate Hyperspheroids. *Eprint Arxiv*, (2014). <https://doi.org/10.48550/arXiv.1403.7664>
- Sun, H.Y., Farooq, M.: Note on the generation of random points uniformly distributed in hyper-ellipsoids. *Proceedings of the Fifth International Conference on Information Fusion I*, 489–496 (2002)
- Gammell, J.D. and T.D. Barfoot.: The Probability Density Function of a Transformation-based Hyperellipsoid Sampling Technique. *Stat.*, (2014). <https://doi.org/10.48550/arXiv.1404.1347>

33. De Ruiter, A.H.J., Forbe, J.R.: On the Solution of Wahba's Problem on $S O(n)$. *Journal of the Astronautical Sciences*. **60**, 1–31 (2014)
34. Guo, G., et al.: Predicting Pareto Dominance in Multi-objective Optimization Using Pattern Recognition. in *Second International Conference on Intelligent System Des. Eng. Appl.* (2012). <https://doi.org/10.1109/ISdea.2012.589>
35. Sanders and Robert: The Pareto Principle: its Use and Abuse. *J. Serv. Mark.* **1**(37), 40 (1987)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Penglei Zhao received the B.S. degree in Electronics and information engineering from Xiamen University, China, in 2019. He is currently pursuing the M.S. degree in the School of Electronic Science and Engineering at Xiamen University. His research interests include path planning, robotic arm control and artificial intelligence.

Yinghui Chang received the M.S. degree and now is a senior engineer. He is mainly engaged in the design and application development of digital SoC integrated circuits. At the same time, he is engaged in the research of reasoning AI chip design and application technology.

Weikang Wu received the B.S. degree in microelectronics from Xidian University in 2011, and the Ph.D. degree in electronics engineering and computer science from Peking University in 2016. Her current interests include computer vision and VLSI radiation-hardened-by-design techniques.

Hongyin Luo Graduated from Xiamen University in 2014 with a doctorate. He once worked in Newland Computer Co., Ltd. and served as the director of Xiamen University-Newland SOC joint laboratory. He was rated as a high-level introduced talent in Fuzhou. In 2018, he founded Xiamen Embedeep Technology Co., Ltd. as the founder and CEO. At present, he is mainly engaged in artificial intelligence, network on chip and integrated circuit design. Since 2013, He has participated in or presided over the research and development of 7 chips, participated in a number of national or provincial or municipal major scientific research projects, published more than 10 papers in domestic and foreign journals, led the research of EEP-TPU IP core technology, and has more than 10 invention patents for core technologies related to artificial intelligence processors.

Zhixin Zhou received the B.S. and M.S. degrees from Xiamen University. He once worked in Xiamen University-Newland SOC joint laboratory and is now the co-founder and hardware technology director of Xiamen Embedeep Technology Co., Ltd. He has participated in the research of 7 chips and has rich practical experience in chip development. He is currently leading the FPGA development and ASIC / SOC chip development of the EEP-TPU IP.

Yanping Qiao received the B.S. and M.S. degrees in Electrical engineering and automation from Guilin University of Electronic Technology, China, in 2013 and 2016, respectively. She is currently pursuing the Ph.D. degree in the School of Electronic Science and Engineering with Xiamen University. Her research interests include neural networks, integrated circuit optimization design and artificial intelligence.

Ying Li received the Bachelor degree from JiangNan University, Wuxi, China, in 2019. She is currently pursuing the Master degree in Xiamen University, Xiamen, China. Her research directions mainly include digital integrated circuit design and evolvable hardware.

Chenhui Zhao received the Bachelor and Master degrees from Shandong University of Science and Technology, Qingdao, China, in 2015 and 2018, respectively. She is currently pursuing the Ph.D. degree in the School of Electronic Science and Engineering at Xiamen University, Xiamen, China. Her research directions mainly include neural networks, evolutionary computation and artificial intelligence.

Zenan Huang received the B.S. degrees in School of Information Science and Technology from Xiamen University, China, in 2016. He is currently pursuing the Ph.D. degree in the School of Electronic Science and Engineering with Xiamen University. His research interests include artificial intelligence and neural network.

Bijing Liu received the Bachelor degree from Hainan University, Hainan, China, in 2016, and graduated with a Master's degree from Xiamen University, Xiamen, China, in 2020. He is currently pursuing the Ph.D. degree in Xiamen University, Xiamen, China. His research directions mainly include artificial intelligence and image processing.

Xiaojie Liu received the B.S. degree in Applied Math from University of California San Diego, USA, in 2014, and the M.S. degree in Electrical and Computer Engineering from Southern Illinois University-Carbondale, USA, in 2019, respectively. She is currently an Assistant Engineer of the School of Electronic Science and Engineering at Xiamen University. Her research areas include artificial intelligence, data analysis, and statistics.

Shan He received the B.S. degree in communication engineering, the M.S. degree in the IC design and verification from Hefei University of Technology, China, in 2004 and 2007, respectively. In 2007, she joined Xiamen University as an Assistant Engineer. She is currently a Senior Engineer of the School of Electronic Science and Engineering. Her research areas include computer networking, artificial intelligence and IC design.

Donghui Guo received the B.S. degree in radio physics, the M.S. and the Ph.D. degrees in semiconductor from Xiamen University, China, in 1988, 1991, and 1994, respectively. In 1994, he joined Xiamen University as a Faculty Member, where he has been a Full Professor, since 2002. He held a post-doctoral, a research fellow, a senior scientist, and a visiting scholar position, respectively, with the City of Hong Kong, University of Ulster, Lawrence Berkeley Laboratory, University California at Berkeley, and University of Illinois at Urbana-Champaign, and served as the Vice-Dean with the School of Information Science and Technology, Xiamen University, from 2008 to 2012. He is currently a senior member of IEEE and the Director of IC Design & IT Research Center. His research areas include computer networking, artificial intelligence, optimization computing, IC design, nano device, and BioMEMS.